

rx.el: Providing s-expression notation for regular expressions



The rx.el Emacs Lisp module provides s-expression notation for regular expressions. It is a macro that can generate regular expressions from readable s-expressions. This eleventh article in the GNU Emacs series explores the various s-expression counterparts and the regular expressions that get generated with numerous examples.

The rx.el Emacs Lisp module is written by Gerd Moellman and is included as part of GNU Emacs, which is released under the GNU General Public License v3.

Installation

The rx.el library is already included as part of GNU Emacs and you can simply *require* the same as shown below:

```
(require 'rx)
```

We will use the s.el package available in Milkypostman's Emacs Lisp Package Archive (MELPA) and in the Marmalede repo to demonstrate the examples. In particular, we will use the *s-matches-p* and *s-match-strings-all* functions provided by the s.el library.

You can install this using the following command inside GNU Emacs:

```
M-x package-install s
```

Alternatively, you can copy the s.el source file to your Emacs load path to use it. If you are using Cask, then you can add the following to your Cask file:

```
(depends-on "s")
```

After installation, you can require the s.el package using the following snippet:

```
(require 's)
```

Usage

Let us now explore the different s-expressions that can be used with the

rx macro to generate regular expressions. The *any* construct matches any character in a SET. The latter may be a character or a string. In the following example '[0-9A-Fa-f]' is the regular expression pattern that is generated.

```
(any SET ...) ;; Syntax
```

```
(setq alphanumeric-pattern (rx (any "a-f" "A-F" "0-9")))
"[0-9A-Fa-f]"
```

```
(s-matches-p (rx (any "a-f" "A-F" "0-9"))
```

```
"A")
```

```
t
(s-matches-p (rx (any "a-f" "A-F" "0-9"))
```

```
";")
```

```
nil
```